ETL Lesson 2 Practice 3: The 4 Extraction Paradigms

🎯 Objective

In this capstone lab, you will step into the role of a Data Engineer to solve a common industry problem: **"How do I get data out of a system?"**

You will implement and compare the four major extraction strategies used in modern data pipelines:

1. **Full Extraction (Snapshot)**
2. **Incremental Extraction (Delta)**
3. **Change Data Capture (CDC)**
4. **Streaming (Event-Based)**

By the end of this session, you will understand the trade-offs of each method regarding data freshness, complexity, and system load.

🏗️ Phase 1: The Setup (Data Acquisition & formatting)

Before we can extract data, we must first acquire it and create a realistic simulation environment.

Real-world production databases change constantly (users update profiles, new orders come in). However, public APIs (like our source, JSONPlaceholder) are **read-only**—they never change. To practice ETL effectively, we must first build a **local "Source System"** that we can modify.

Step 1: Fetching Raw Data from the API

The Goal: Connect to the external API and retrieve the raw user directory.

1. **Connect:** Your script will send a `GET` request to `https://jsonplaceholder.typicode.com/users`.
2. **Receive:** The API responds with a payload in **JSON (JavaScript Object Notation)** format.
3. **Inspect:** You will see a list of 10 user objects.
    - *Challenge:* The data is **nested**. For example, a user's city is hidden inside an `address` dictionary (`user['address']['city']`), and their company name is inside a `company` dictionary.
    - *Requirement:* Most analytical databases (like SQL or Data Warehouses) prefer **flat** tables, not nested JSON.

Step 2: Data Flattening & Normalization

The Goal: Convert the complex, nested JSON into a flat, tabular format suitable for a database.

1. **Parse:** Iterate through the list of users.
2. **Flatten:** For each user, extract specific fields to the top level.
    - *Example:* Move address.city $\rightarrow$ city.
    - *Example:* Move company.name $\rightarrow$ company_name.
3. **Select:** Discard unnecessary technical fields (like lat, long, or catchPhrase) to clean the dataset.
4. **Format:** Ensure data types are consistent (e.g., ensuring IDs are integers, emails are strings).

Step 3: Building the "Source" Database

The Goal: Create a local SQLite database to act as our "Production System."

1. **Initialize DB:** Create a local file named production_source.db.
2. **Create Schema:** Define a table named users with columns that match your flattened data (id, name, email, city, company).
3. **Add Watermarking Columns:**
    - **Crucial Step:** Add a column named updated_at. This is essential for Incremental Extraction. It tells us *when* a row was created or last modified.
4. **Load Initial Data:** Insert the 10 flattened user records into this table. Set their updated_at timestamp to "Yesterday" (so they appear as existing historical data).

Step 4: Simulating "Business Activity"

The Goal: Mimic real-world users interacting with the system so our ETL pipelines have something new to find.

1. **Simulate an UPDATE:**
    - *Action:* Change User #1's email address.
    - *System Event:* Update the email column and refresh the updated_at timestamp to "Now."
2. **Simulate an INSERT:**
    - *Action:* Create a brand new user (User #99).
    - *System Event:* Add a new row with the updated_at timestamp set to "Now."
3. **Simulate a DELETE:**
    - *Action:* Delete User #2.
    - *System Event:* Remove the row entirely from the users table.
    - *CDC Logging:* (Advanced) Simultaneously write a "DELETE" record to a separate cdc_log table so we have a record that this user *used* to exist.

🚀 Phase 2: Executing the 4 Extraction Strategies

Now that our source database has "changed" (it differs from yesterday's state), we will run four different pipelines to see how they handle these changes.

Method A: Full Extraction (Snapshot)

The "Brute Force" Approach

1. **Logic:** "Give me everything you have right now."
2. **Process:**
    - Connect to the `users` table.
    - Select **all rows** (`SELECT *`).
    - Save the result to `users_full.csv`.
3. **Observation:**
    - You will see the *new* user.
    - You will see the *updated* email.
    - **CRITICAL:** You will *not* see User #2. They are just gone. You have no record that they were ever there.

Method B: Incremental Extraction (Watermarking)

The "Smart Batch" Approach

1. **Logic:** "Give me only what changed since the last time I checked."
2. **Process:**
    - Define a **High Watermark**: This is the timestamp of your last successful run (simulated as "Yesterday").
    - Query the database: `SELECT * FROM users WHERE updated_at > Last_Watermark`.
    - Save the result to `users_incremental.csv`.
3. **Observation:**
    - You will get a very small dataset (only the updated user and the new user).
    - **CRITICAL:** You will miss the DELETE event entirely. Since the row for User #2 was deleted, it has no timestamp to check! This reveals a major weakness of standard incremental loading.

Method C: Change Data Capture (CDC)

The "Forensic" Approach

1. **Logic:** "Give me the transaction logs, not the current table state."
2. **Process:**
    - Instead of reading the `users` table, connect to the `cdc_log` table (simulated in Phase 1).
    - Extract all events tagged with a timestamp later than "Yesterday."
    - Save to `users_cdc.csv`.

3. **Observation:**
    - You will see three distinct rows:
        1. OP: UPDATE for User 1.
        2. OP: INSERT for User 99.
        3. OP: DELETE for User 2.
    - This is the only batch method that accurately captures deletions.

Method D: Streaming (Push Notifications)

The "Real-Time" Approach

1. **Logic:** "Don't call us, we'll call you."
2. **Process:**
    - **The Producer:** We will start a background process that generates random events (simulating users clicking buttons or logging in) and pushes them into a "Queue."
    - **The Consumer:** Your main script will "listen" to this queue. As soon as a message arrives, it prints it out or saves it.
3. **Observation:**
    - Unlike the previous three methods which looked at the *Database*, this method looks at *Activity*.
    - You will capture transient events (like "User Login") that might never even be written to a database table.

📊 Phase 3: Analysis & Comparison

Finally, you will compare the output files to answer these questions:

1. **Volume:** Which method extracted the most rows? (Hint: Full Extraction is always the heaviest).
2. **Completeness:** Which method failed to detect the deleted user?
3. **Latency:** Which method would be best if I needed to know about a new signup within 5 seconds?

You will produce a Python Notebook that performs these steps and generates a visualization (Bar Chart) comparing the number of records processed by each method.